

JavaScript Base – Vue 3 (SPIEGAZIONE + ESERCIZI + BACKTICK)

0. COME SI FA IL BACKTICK (`)

Il **backtick** è il simbolo usato nei template literals.

👉 È questo: 

Windows (tastiera italiana)

Metodo 1 (più comune)

ALT GR + tasto vicino al numero 1

Quel tasto è spesso:

- 
- 
- 
- 

👉 Premi **ALT GR** + quel tasto

Metodo 2 (se non funziona)

ALT + 96 (dal tastierino numerico)

Linux (tastiera italiana)

Metodo 1

ALT GR + tasto vicino a 1

Metodo 2

Dipende dal layout:

Compose + `

! IMPORTANTE

NON confondere:

simbolo	nome
`	backtick
'	apice singolo
"	doppio apice

💡 ESEMPIO TEMPLATE LITERAL

```
``javascript id="btck1" let nome = "Mario";
```

```
console.log( Ciao ${nome} );
```

```
---  
  
# 1. DEFINIZIONI BASE (SEMPLIFICATE)  
  
---  
  
## let  
Variabile modificabile.  
  
``javascript id="let01"  
let nome = "Mario";  
nome = "Luigi";
```

const

Variabile fissa.

```
``javascript id="const01" const PI = 3.14;
```

```
---  
  
## String  
Testo.
```

```
````javascript id="str01"
let nome = "Mario";
```

---

## Number

Numeri.

```
````javascript id="num01" let eta = 30;
```

```
---

## Boolean
true / false.

````javascript id="bool01"
let attivo = true;
```

---

## Template Literals

Stringhe con backtick.

👉 servono per inserire variabili.

```
````javascript id="temp01"
console.log( Ciao ${nome});
```

```
---

## Arrow Function
Funzione corta.

````javascript id="arrow01"
const doppio = n => n * 2;
```

---

## Destrutturazione

Prendi valori da oggetto.

```
````javascript id="des01" const { nome, prezzo } = prodotto;
```

2. SPREAD OPERATOR (SUPER SEMPLICE)

👉 significa: COPIA TUTTO

Oggetti

```
```javascript id="sp01"
const prodotto = { nome: "Laptop", prezzo: 999 };
const copia = { ...prodotto };
```

---

## Modifica

```
```javascript id="sp02" const nuovo = { ...prodotto, prezzo: 899 };
```

Unione

```
```javascript id="sp03"
const extra = { garanzia: 24 };
const completo = { ...prodotto, ...extra };
```

---

## Array

```
```javascript id="sp04" const numeri = [1, 2, 3]; const copia = [...numeri];
```

3. ARRAY METHODS

- map → cambia
- filter → seleziona
- reduce → somma
- find → trova

4. ASYNC / AWAIT

```
```javascript id="async01"
```

```
async function f() {
 const res = await fetch(url);
}
```

---

## 5. ESERCIZI COMPLETI (CON SPIEGAZIONE)

---

### ESERCIZIO 1

#### 👉 COSA FAI

Crei variabili e stampi una frase.

```
````javascript id="ex1" let nome = "Mario"; let eta = 30; let attivo = true;
```

```
console.log(Utente: ${nome}, ${eta} anni, stato: ${attivo ? "attivo" : "non  
attivo"});
```

```
const PI = 3.14; // PI = 3.15 ✖ ERRORE
```

```
---  
  
# ESERCIZIO 2  
  
## 👉 COSA FAI  
Funzioni veloci.  
  
````javascript id="ex2"  
const raddoppia = n => n * 2;

const saluta = (nome, ora) =>
 ora < 12 ? `Buongiorno ${nome}` : `Buonasera ${nome}`;
```

---

### ESERCIZIO 3

#### 👉 COSA FAI

Oggetto → copia → modifica.

```
````javascript id="ex3" const prodotto = { nome: "Laptop", prezzo: 999, marca: "Dell", disponibile: true };
```

```
const { nome, prezzo } = prodotto;
```

```
const aggiornato = { ...prodotto, prezzo: 899 };
```

```
const completo = { ...prodotto, garanzia: 24, colore: "grigio" };
```

```
---
```

```
# ESERCIZIO 4
```

```
## 👉 COSA FAI
```

```
Lavori su lista di prodotti.
```

```
```javascript id="ex4"
```

```
const prodotti = [
 { nome: "Laptop", prezzo: 999, categoria: "tech" },
 { nome: "Mouse", prezzo: 29, categoria: "tech" },
 { nome: "Scrivania", prezzo: 350, categoria: "arredo" },
 { nome: "Monitor", prezzo: 450, categoria: "tech" }
];
```

```
const nomi = prodotti.map(p => p.nome);
```

```
const tech = prodotti.filter(p => p.categoria === "tech" && p.prezzo > 100);
```

```
const totale = prodotti.reduce((a, p) => a + p.prezzo, 0);
```

```
const mouse = prodotti.find(p => p.nome === "Mouse");
```

---

## ESERCIZIO 5

### 👉 COSA FAI

Prendi dati da internet.

```
```javascript id="ex5" async function todo() { try { const res = await fetch("https://  
jsonplaceholder.typicode.com/todos/1"); const data = await res.json(); console.log(data.title); } catch (err)  
{ console.log(err); } }
```

```
async function todos() { try { const res = await fetch("https://jsonplaceholder.typicode.com/todos?  
_limit=5"); const data = await res.json();
```

```
const completati = data.filter(t => t.completed);  
console.log(completati);
```

```
} catch (err) { console.log(err); } } ````
```

RIASSUNTO FINALE

- let = cambia
- const = fisso
- backtick = ` → serve per \${}
- spread = copia tutto
- map = trasforma
- filter = seleziona
- reduce = somma
- find = trova
- async = internet lento